IFC-ECS: A simplified IFC ECS approach facilitating IFC4.X migration

February 22, 2026

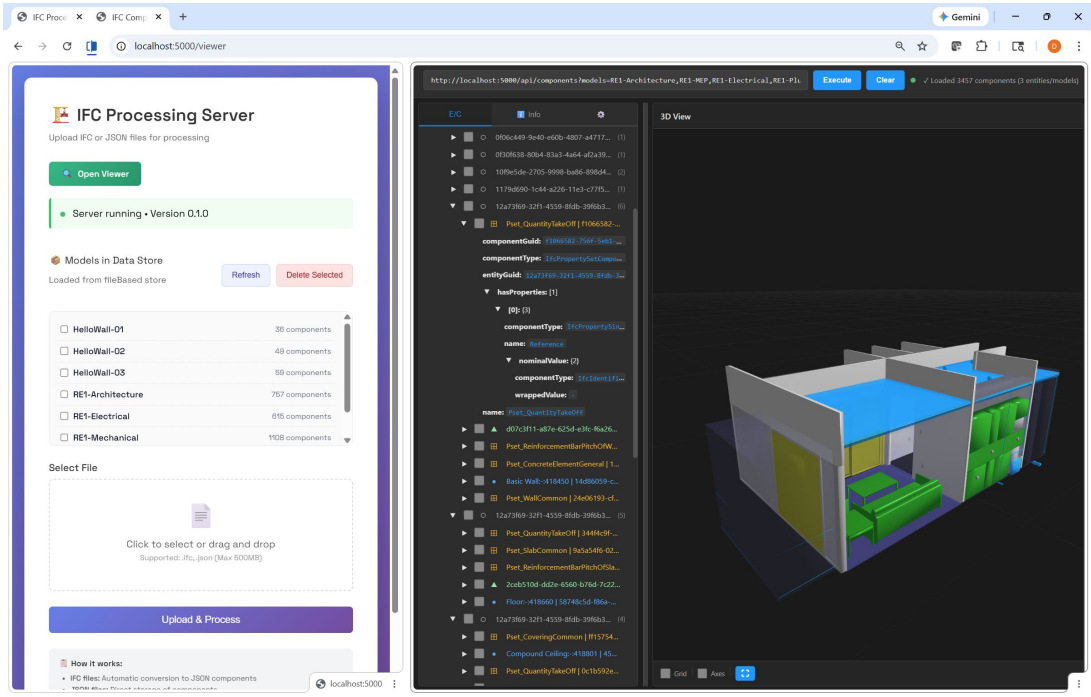
This document introduces an approach to IFC modernization based on the ECS design pattern. The core was developed during the Exton implementer's meeting in the API group, and is being developed into a suite of data strategies, standards definitions, and reference implementations. Major elements of this development include:

1. A simple data convention for migrating V4 class instances to entities and components.
2. An IFC2.3 - IFC 4.0 converter, which generates Json IFC-ECS based components from IFC files.
3. An initial API proposal
4. A reference implementation component server that ingests legacy (2.3 – 4.0) IFC files, stores them in a local file store (so that eventually they can be gitt'ed), reads them into memory and provides API end points and query parameters.
5. A web viewer which allows users to issue queries to servers, and navigate results.

A limited running version of the server is running (models aren't stored and need to be uploaded) https://ifc-ecs-prototype-daa447b4d672.herokuapp.com/

The full repo is https://github.com/Drshelden/IFCXPrototype. To run:

- set up the environment (currently .venv\Scripts\Activate.ps1 or possibly activate or activate.bat)
- run python server.py
- upload some ifc models at http://localhost:5000/
- open the viewer
- type some queries in like http://localhost:5000/api/components?models=HelloWall-03&entityTypes=IfcDoor



This document describes the initial assumptions and exploratory implementations and possible further developments.

I. Component data format

The approach proposes a simple initial migration of IFC4X class instance data components, and a way of promoting certain IFC IDs to either entity or component Guids. Here we consider a few key class types and the migration approach, and initial tests suggest the approach can be extended to all other IFC classes. As a general strategy:

- IFC instances are converted components, and their attributes or parameters become key / value pairs in the component dictionary. A few additional entries are added, specifically the **componentGuid, componentType, entityGuid**, and in certain cases an **entityType** that informs the system of the object type associated with the entityGuid. The IFCv4 ID is used in some cases (specifically object definitions) as the entityGuid, and in others as the componentGuid, with the entityGuid determined based on the instance type.
- Objects are selectively passed either by reference or value. As an initial rule, objects with a GlobalId are passed by reference and are "First Class Objects", all others by value.

Object definitions

Here is a typical IFC building element definition from BSDD with the additional EC attributes and its representation as a component in Json:

Name	Data type	Units	Definition	Dictionary	Identifier (URI)
Attributes					
EntityGuid					
EntityType					
ComponentGuid					
ComponentType					
Description	String		Optional description, provided for exchanging informative comments.	IFC	
Element Type	String		The type denotes a particular type that indicates the object further. The use has to be established ...	IFC	
Global ID	String		Assignment of a globally unique identifier within the entire software world.	IFC	
Name	String		Optional name for use by the participating software systems or users. For some subtypes of IfcRoot...	IFC	
Object Type	String		The type denotes a particular type that indicates the object further. The use has to be established ...	IFC	
Operation Type	String		Type of operations. Also used to assign standard symbolic presentations according to national bu...	IFC	
Overall Height	Real		Overall measure of the height, it reflects the Z Dimension of a bounding box, enclosing the body of ...	IFC	
Overall Width	Real		Overall measure of the width, it reflects the X Dimension of a bounding box, enclosing the body of t...	IFC	


```
{
  "componentGuid": "5c890f20-ee21-063a-7d35-3cffdf73b494",
  "componentType": "IfcDoorComponent",
  "entityGuid": "f1c8fc3e-82ee-4c75-ae69-2b1be0926228",
  "entityType": "IfcDoor",
  "name": "Door-Passage-Single-Flush:36\" x 84\":1240792",
  "objectType": "Door-Passage-Single-Flush:36\" x 84\"",
  "overallHeight": 7.333333333333333,
  "overallWidth": 3.333333333333333,
  "tag": "1240792"
}
```


Some of the key assumptions – very much open for debate:

- The Global ID becomes the entityGuid for this component. Other components related to this entity reference this entityGuid (not one computed for them)
- A new, unique and deterministic computed Guid is created for the componentGuid.
- The V4 IFC class becomes the entityType. So far Object Definition components are unique in declaring this entityType, and these components inform the system of the type associated with the entity.
- Each component has a type, which is simply the V4 type with 'Component' concatenated.
- All parameter names are converted to lowerCamelCase.

- The GlobalId and a few other attributes are removed. The list at right shows those excluded in the current converter implementation, in some cases to cull down the initially exported information pending further standardization decisions, and can be re-introduced later.

```
    'ownerhistory',
    'id',
    'step_id',
    'objectplacement',
    'representation',
    'representations',
    'representationmaps',
    'representationcontexts',
    'unitsincontext',
    'globalId'
}
```

PropertySets

PropertySets are treated similarly, except their entityGuid references that of the object definition entity defined above, and the PropertySet's GlobalId becomes its componentGuid. Like most components, PropertySetComponents do not include an entityType parameter.

Name	Data type	Units	Definition	Dictionary	Identifier (URI)
Attributes					
EntityGuid					
ComponentGuid					
ComponentType					
Pset_DoorCommon					
Acoustic Rating	String		Acoustic rating for this object.; It is provided according to the national building code. It indicat...	IFC	
Durability Rating	String		Durability against mechanical stress. It is given according to the national code or regulation.	IFC	
Fire Exit	Boolean		Indication whether this object is designed to serve as an exit in the case of fire (TRUE) or not (FA...	IFC	
Fire Rating	String		Fire rating for this object. It is given according to the national fire safety classification.	IFC	
Glazing Area Fraction	Real		Fraction of the glazing area relative to the total area of the filling element.; It shall be used, i...	IFC	
Handicap Accessible	Boolean		Indication that this object is designed to be accessible by the handicapped. Set to (TRUE) if this o...	IFC	
Has Drive	Boolean		Indication whether this object has an automatic drive to operate it (TRUE) or no drive (FALSE)	IFC	

```
{
  "componentGuid": "6d533265-65f4-fc00-e847-7e73cc3ab3f7",
  "componentType": "IfcPropertySetComponent",
  "entityGuid": "f1c8fc3e-82ee-4c75-ae69-2b1be0926228",
  "hasProperties": [
    {
      "componentType": "IfcPropertySingleValueComponent",
      "name": "Reference",
      "nominalValue": {
        "componentType": "IfcIdentifierComponent",
        "wrappedValue": "36\" x 84\""
      }
    }
  ],
  "name": "Pset_QuantityTakeOff"
}
```

<u>IfcRelations</u>

There are some design decisions about how to treat entities associated with IfcRelations. The current attempt derives the entityGuid from the GlobalId of the Relating object if it exists, otherwise there is <u>no</u> entityGuid, and the relating object is expanded into the Relating parameter's value. Below are examples of a relationship where the Relating object is promoted to the entityGuid, and another where the Relating object isn't first class and is expanded.

```
{
  "componentGuid": "933c4a06-93b8-11d3-80f8-00c14f8efc2c",
  "componentType": "IfcRelContainedInSpatialStructureComponent",
  "entityGuid": "933c4a06-93b8-11d3-80f8-00c04f8efc2c",
  "relatedElements": [
    "f1c8fc3e-82ee-4c75-ae69-2b1be09262cd",
    "f1c8fc3e-82ee-4c75-ae69-2b1be0926297",
    "f1c8fc3e-82ee-4c75-ae69-2b1be0926228"
  ]
},
```

```
{
  "componentGuid": "5ee46b9f-a508-fc7b-3fde-e8df6c667753",
  "componentType": "IfcRelAssociatesMaterialComponent",
  "relatedObjects": [
    "0d68ad35-29c7-484d-907c-65a3fbc0eadb"
  ],
  "relatingMaterial": {
    "componentType": "IfcMaterialLayerSetUsageComponent",
    "directionSense": "NEGATIVE",
    "forLayerSet": {
      "componentType": "IfcMaterialLayerSetComponent",
      "layerSetName": "Basic Wall:Generic - 8\"",
      "materialLayers": [
        {
          "componentType": "IfcMaterialLayerComponent",
          "layerThickness": 0.6666666666666666,
          "material": {
            "componentType": "IfcMaterialComponent",
            "name": "Default Wall"
          }
        }
      ]
    },
    "layerSetDirection": "AXIS2",
    "offsetFromReferenceLine": 0.3333333333333333
  }
},
```

ShapeRepresentations

In the current implementation ShapeRepresentations are transformed so that they refer to the entityGuid of the object they represent. The downstream geometry is transformed into a mesh and the verbose geometry is removed (for now). The mesh is currently represented in OBJ format in global coordinates. We anticipate potentially moving to .gltf or .glb, for meshes and revisiting parametric geometry representations in future implementations.


```
{
  "componentGuid": "cbb1b9d8-b5b8-d604-1715-c5157acc3229",
  "componentType": "IfcShapeRepresentationComponent",
  "entityGuid": "f1c8fc3e-82ee-4c75-ae69-2b1be0926228",
  "representationIdentifier": "Body",
  "representationFormat": "OBJ",
  "items": [
    "v -2.01486169281188 0.023749999999998328 1.0089289321881372\n
    f 13 2 3\nf 13 3 4\nf 13 4 5\nf 13 5 6\nf 13 6 7\nf 13 7 8\n"
  ]
}
```


Some open questions:

- *Is there a 3rd case besides First Class objects (with GlobalIds) and others that are passed by value? Should other 4X object classes be included as referenceable objects and therefore receive persistent componentGuids?*
- *What types of 4.X objects should be promoted to Entities?*
- *Can components refer to components via their componentGuids? This question emerges in the case of IfcRelations*
- *Which 4X classes should be included in the initial core, and how generally should classes be organized into modules.*
- *Other refinements of the existing 4X object classes and definitions as we go…*

II. Data converter

A utility that converts legacy IFC to IFC-ECS has been developed. The approach is greatly informed by the IFCJSON converter by Jan Brauer, and heavily uses IfcOpenShell. The utility can be run from the command line and is also used to ingest IFC4.X files into the server. To run:

```
python ifc4ingestor.py inputFile.ifc -o outputFile.json.
```

Note that currently the assimilator only saves components for subclasses of IfcObjectDescription, IfcPropertySet, IfcRelation, and IfcShapeRepresentation. The set of classes and parameters supported in this version is deliberately culled to remove many of the subsets of IFC objects:

- all geometries are stripped out and replaced with a triangulaged IfcShapeRepresentation,
- miscellaneous classes like Actors, Scheduled etc. are not included yet (although the current importer should generally support them - needs more testing)
- Others are filtered out simply by not including them on import (in which case they are either imported by value or simply left pointing to the IFC Guid of the relevant entity)

- "First Class entities" (with a GlobalId) are referred to by guid reference whether they are included in the export or not (ie, whether they are subclasses of IfcObjectDescription, IfcPropertySet, IfcRelation, and IfcShapeRepresentation), or are expanded as values in other cases.

III. API

A simple REST API has been proposed and is implemented in the demonstration server, below. This API provides simple queries to find entity and component references and their values. The set of rest end points is documented in this API_DOCUMENTATION, API_QUICK_REFERENCE and this openapi.yaml file. The concept of a model is provisionally introduced, a model equates to a given uploaded file and its name is derived from the file name (to be discussed)

End points include:

- `models` `#lists the models on the server`
- `entityTypes` `# lists the types of entities in the model`
- `entityGuids` `#lists the entityGuids`
- `componentGuids` `#lists the componentGuids`
- `components` `#returns component data`

Wherever appropriate the following filters can be applied:

- `models=model1,model2,...`
- `entityTypes=IfcType1,IfcType2,...`
- `componentTypes=IfcType1Component,IfcType2Component,...`
- `entityGuids=guid1,guid2,...`
- `componentGuids=guid1,guid2,...`

In the proposed API and reference implementation, for entityTypes and componentTypes filters you can enter a base class like IfcProduct or IfcProductComponent and the returned values will include entires with that base and its subclasses.

Example queries:

`http://localhost:5000/api/entityGuids?models=HelloWall-01,HelloWall-02`

`http://localhost:5000/api/componentGuids?entityTypes=IfcProduct`

`http://localhost:5000/api/components?componentTypes=IfcRelAggregatesComponent`

`http://localhost:5000/api/components?componentTypes=IfcRelAggregatesComponent`

`http://localhost:5000/api/components?componentGuids=156d4a9b-8c6b-d7dd-d6fc-b0dabd0b3d9c,8fcce169-98e4-9471-53dd-a583112814c7`

An OpenAI .yaml file has been created to document the API

```yaml
/api/entityGuids:
    get:
      tags:
        - Queries
      summary: Query for entity GUIDs
      description: |
        Query entity GUIDs based on optional filters.
        Results are organized by model name.
        - Filter by model name(s) (optional)
        - Filter by entity type(s) (optional)
      operationId: queryEntityGuids
      parameters:
        - name: models
          in: query
          description: Comma-separated list of model names to filter by
                          (optional - includes all models if omitted)
          schema:
            type: string
            example: HelloWallIFCjsonC-2x3
        - name: entityTypes
          in: query
          description: Comma-separated list of entity types to filter by
          schema:
            type: string
            example: IfcWall,IfcPropertySet
      responses:
        '200':
          description: Dictionary mapping model names to arrays of entity GUIDs
          content:
            application/json:
              schema:
                type: object
                additionalProperties:
                  type: array
                  items:
                    type: string
                example:
                  HelloWallIFCjsonC-2x3:
                    - entity-guid-1
                    - entity-guid-2
                    - entity-guid-3
        '400':
          description: Invalid query parameters
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
```

Some open questions:

- *The models concept was introduced for expediency. Does it or any container convention make sense, if so what should it be?*
- *This API sticks closely to basic open world Entity/Component concepts. One limitation of this approach is the need to continually search for pointing entities upstream from relationships. Within a given data store, alternative query languages maybe based on the IfcOpenShell API or the product structure would greatly simply model retrieval and navigation.*

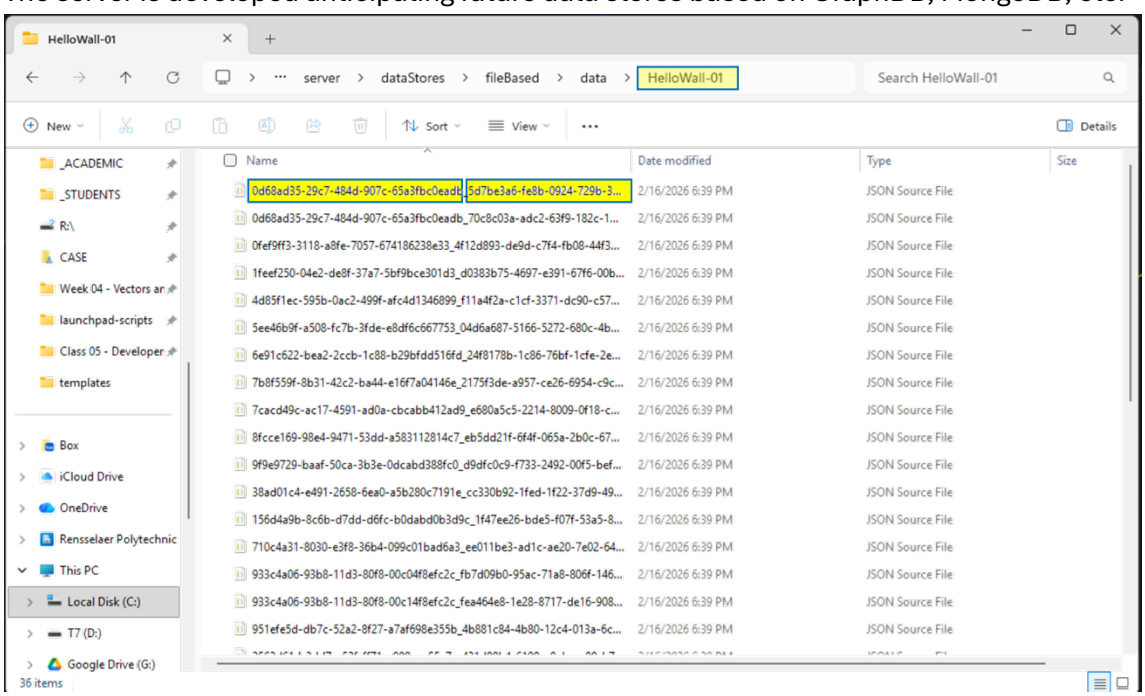
IV. Component Server

A demonstration component server implementing the API has been developed. This server does the following:

- **Allows users to upload .ifc files** which are converted to components by the IFC4.X converter, (or uploaded in IFCXJson directly). The file name minus its type is used as the model name.

- **Stores the components locally.** Currently every component is written out to a folder with the name "entityGuid_componentGuid.json", in the folder named by the file name of the uploaded model.ifc file. By storing this way we anticipate git-like interactions with this store. Note also that components associated with the same entity are grouped together alphabetically.

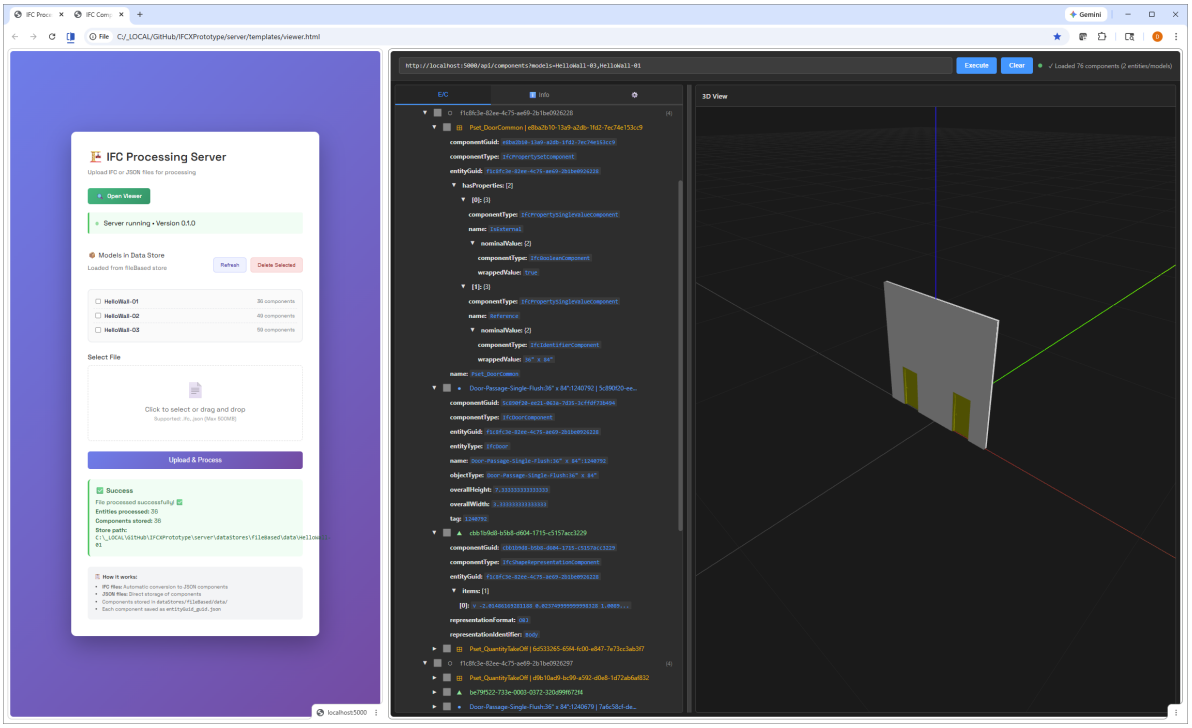
  The server is developed anticipating future data stores based on GraphDB, MongoDB, etc.



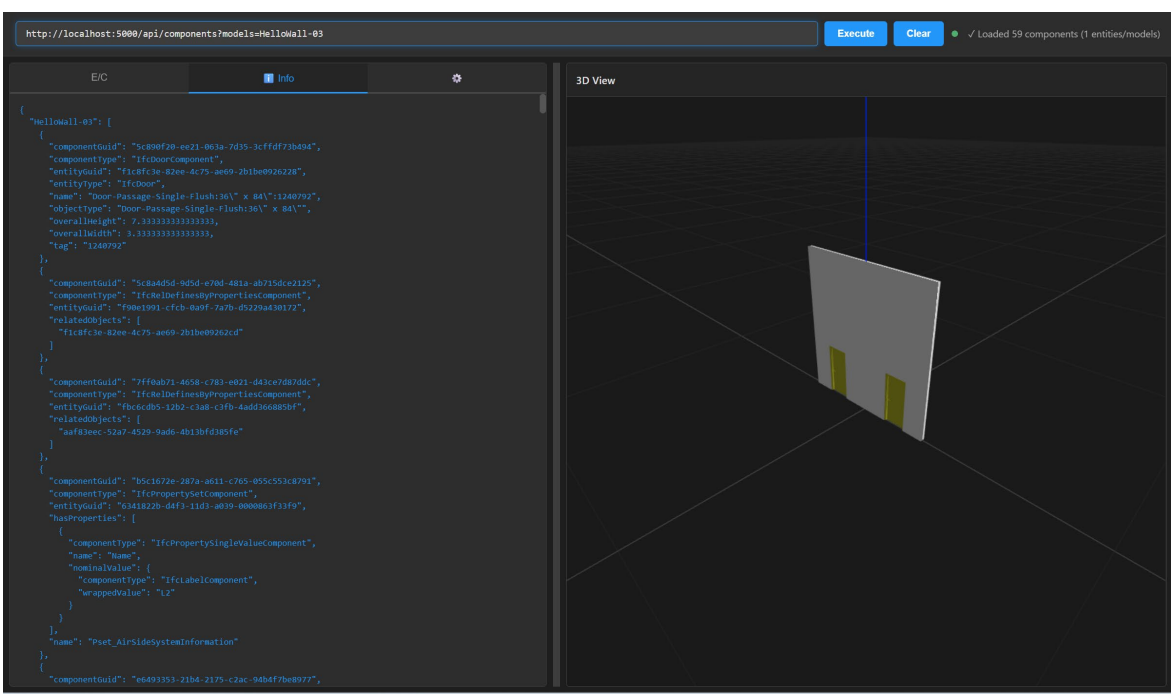
- **Allows retrieval by REST end point queries**, as discussed in III above.

V. Viewer

A simple Web app is being developed to demonstrate the API query functionality, Entity-Component organization and allow viewing of both component metadata and ShapeRepresentation meshes. Server admin view and viewer are shown below.



In addition to the Entity/Component tree, an Info tab shows the exactly the returned results from the query.



VI. Further work

The described prototype has been developed to facilitate discussion about a candidate IFCX data architecture and data migration strategy, and to allow demonstration and testing of assumptions. There are many preliminary assumptions and proposals that need to be considered and tested before adoption, and the current implementation suggests many areas for further work. A few are listed below. However, adopting a simplified strategy similar to the one discussed could allow rapid progression through decision making, testing and exploration by the BuildingSmart and IFCX communities.

Further topics:

- **Entity and component names and name spaces.** The current approach uses entityType and componentType as simple names, but no indication of their provenance is provided. An approach using URIs pointing to components' definitions was suggested, and / or BSI could develop a name registry service a la pip, npm, etc.
- **Schema modularization.** Json Schema provides useful mechanisms for portioning and merging sub-schemas.
- **Types and typicals.** An approach based on a simple inheritsFrom or extends parameter on components has been suggested. USD may provide some valuable precedent.
- **Git**-like functionality has been proposed as the means for versioning, permissions, "work sets", etc. How would this work?
- **Transmission details** – metadata wrappers for transmitted packages of components has been proposed. Cloud events is a fairly widely adopted way of doing this.
- **Class hierarchy and parameter simplification.**
- **External referencing of useful 3rd party ontologies,** such as QUDT, etc.
- **Alternative encodings beyond Json and REST**, including JSON-LD, Graph-GL, SHACL, etc.
- *Many others…*